

Account Management System

Object-Oriented Programming — CCC 1223

Semester 2, Year 1 (2025/2026)

Md Ismayel Shekh — AIU24102132
Kabisha Hafizi — AIU24102394
Md Towhidul Islam Tamim Mia — AIU24102441
Md Ariful Islam — AIU24102442



Group A

Lecturer: Siti Shafrah Shahawai

TABLE OF CONTENTS

01 Introduction & Problem Description

02 System Design & Class Architecture

03 OOP Concepts — Inheritance

04 OOP Concepts — Polymorphism

05 System Features & GUI Screens

06 Implementation Details

07 Sample Input & Output

08 System Flowchart

09 Conclusion

01 | INTRODUCTION

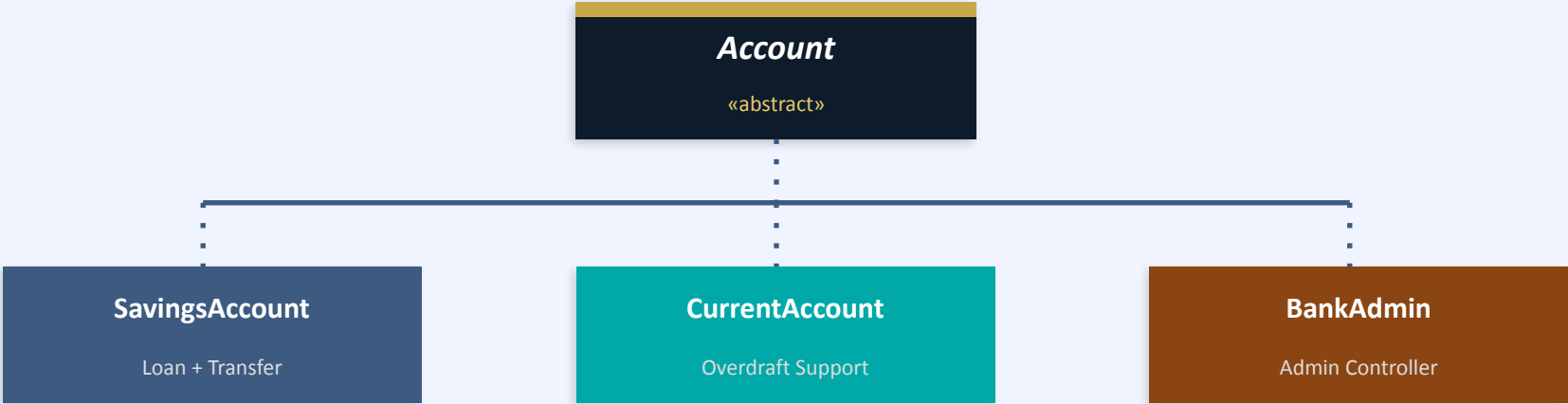
Problem Description

- ▶ Traditional banking requires physical attendance
- ▶ Small-scale simulations lack proper OOP design
- ▶ Difficult to maintain & extend existing systems
- ▶ No structured digital banking interface

Objectives

- ▶ Develop a working Java GUI banking app
- ▶ Apply OOP — inheritance, polymorphism, encapsulation
- ▶ Implement user-friendly Swing-based interface
- ▶ Manage multiple user accounts using ArrayList
- ▶ Separate roles: Account, SavingsAccount, CurrentAccount, BankAdmin

02 | SYSTEM DESIGN & CLASS ARCHITECTURE



Key Attributes & Methods — Account (Abstract Parent)

accountHolderName	String	Full name of account holder
accountNumber	int	Auto-generated 6-digit unique ID
balance	double	Current account balance
deposit(amount) / deposit(amount, note)	Method (Overloaded)	Add funds — plain or with label
withdraw(amount)	Method	Deduct funds if balance sufficient
getAccountType()	Abstract	Overridden by child classes

03

INHERITANCE

SavingsAccount and CurrentAccount both extend Account using the extends keyword, automatically inheriting all attributes and methods.



SavingsAccount.java

```
class SavingsAccount extends Account {  
    private int loanCount;  
    private boolean loanEnabled;  
  
    public SavingsAccount(String name) {  
        super(name); // calls Account()  
        this.loanCount = 0;  
        this.loanEnabled = true;  
    }  
}
```

Class	Parent	Relationship
Account	—	Abstract parent class
SavingsAccount	Account	Inherits all + adds loan/transfer
CurrentAccount	Account	Inherits all + overrides withdraw

Inherited attributes: accountHolderName | accountNumber | balance | transactionHistory

Inherited methods: deposit() | withdraw() | logTransaction() | getTransactionHistory()

04 | POLYMORPHISM

METHOD OVERRIDING — Runtime Polymorphism

```
// In Account (abstract)
public abstract String getAccountType();

// SavingsAccount
@Override
public String getAccountType() {
    return
    "Savings Account";
}

// CurrentAccount (overdraft)
@Override
public boolean withdraw(double amount) {
    if (amount > balance+overdraft)
        return false;
    balance -= amount;
    logTransaction(...);
    return true;
}
```

METHOD OVERLOADING — Compile-time Polymorphism

```
// Overload 1 — basic deposit
public boolean deposit(double amount) {
    if (amount <= 0) return false;
    balance += amount;
    logTransaction(
    "DEPOSIT", amount);
    return true;
}

// Overload 2 — deposit with note
public boolean deposit(double amount, String note) {
    if (amount <= 0) return false;
    balance += amount;
    logTransaction(
    "DEPOSIT [" + note + "]", amount);
    return true;
}
```

05 | SYSTEM FEATURES & GUI SCREENS



User Registration

Create Savings Account with username & password



User Login

Validates credentials via BankAdmin HashMap lookup



Deposit / Withdraw

Input validated; balance updated with timestamp log



Fund Transfer

Atomically adjusts sender & recipient balances



Transaction History

Timestamped log in scrollable JTextArea



Take Loan

Up to 2 loans; checks reserve & loanEnabled flag



Admin Panel

Create/delete accounts, view all, toggle loan, stats



GUI — 6 Screens

CardLayout: Welcome, Login, Register, Dashboard, Admin Login, Admin Panel

06 | IMPLEMENTATION DETAILS

Data Structures



`ArrayList<Account>` — `allAccounts`
(runtime add/remove)



`ArrayList<SavingsAccount>` —
`savingsAccounts` for transfers



`HashMap<Integer, String>` — $O(1)$
password lookup by account number



`ArrayList<String>` — per-account
transaction history with timestamps

Control Structures



if-else: `deposit > 0`, balance check for
withdrawal



if-else: `loanEnabled && loanCount < 2`



if-else: overdraft — `amount ≤ balance + 500`



for-each: `findAccount()`, `deleteAccount()`,
`getAllAccounts()`, `toggleLoan()`

GUI Architecture



`JFrame` + `CardLayout` — 6 screen panels



`JOptionPane` — all input prompts & result
dialogs



`JTextArea` + `JScrollPane` — transaction
history display



`GridBagLayout` & `GridLayout` — form and
button grids

07 | SAMPLE INPUT & OUTPUT

Sample User Session		
1	Register	Ahmad Zaki zaki1234
2	Login	Ahmad Zaki zaki1234
3	Deposit	RM 500.00
4	Withdraw	RM 150.00
5	Transfer	To Nur Aisyah → RM 100.00
6	Take Loan	RM 200.00
7	History	(click History button)

Transaction History Output

1. [30/05/2026 10:05:22] DEPOSIT [ATM]
: RM 500.00 | Bal: RM 500.00

2. [30/05/2026 10:06:11] WITHDRAWAL
: RM 150.00 | Bal: RM 350.00

3. [30/05/2026 10:07:03] TRANSFER TO Nur Aisyah
: RM 100.00 | Bal: RM 250.00

4. [30/05/2026 10:08:44] LOAN RECEIVED
: RM 200.00 | Bal: RM 450.00

Live Dashboard Balance: RM 450.00

Validation checks ensure positive amounts, sufficient balance, recipient existence, loan limits, and correct password inputs.

08 | SYSTEM FLOWCHART

A. User Login Flow

User clicks Login

Enter username & password

BankAdmin.findAccount()
iterates ArrayList

Account found?

Set currentUser
Refresh Dashboard

Show error message

B. User Dashboard Flow

Show Dashboard

Choose action:
Deposit / Withdraw
Transfer / Loan

Validate input (if-else)

Call method on
SavingsAccount object

logTransaction()
with timestamp

Refresh balance label

C. Admin Flow

Admin Login

BankAdmin.authenticate()

Show Admin Panel

Choose operation:
Create / Delete / View
Toggle Loan / Stats

Update ArrayList
or flags directly

Logout → Welcome

OOP CONCEPTS SUMMARY



Inheritance

- ✓ Account is abstract parent class
- ✓ SavingsAccount extends Account
- ✓ CurrentAccount extends Account
- ✓ super() calls parent constructor



Polymorphism

- ✓ Overriding: getAccountType() in both children
- ✓ Overriding: withdraw() in CurrentAccount
- ✓ Overloading: deposit(amount) & deposit(amount,note)
- ✓ Runtime vs compile-time polymorphism



Encapsulation

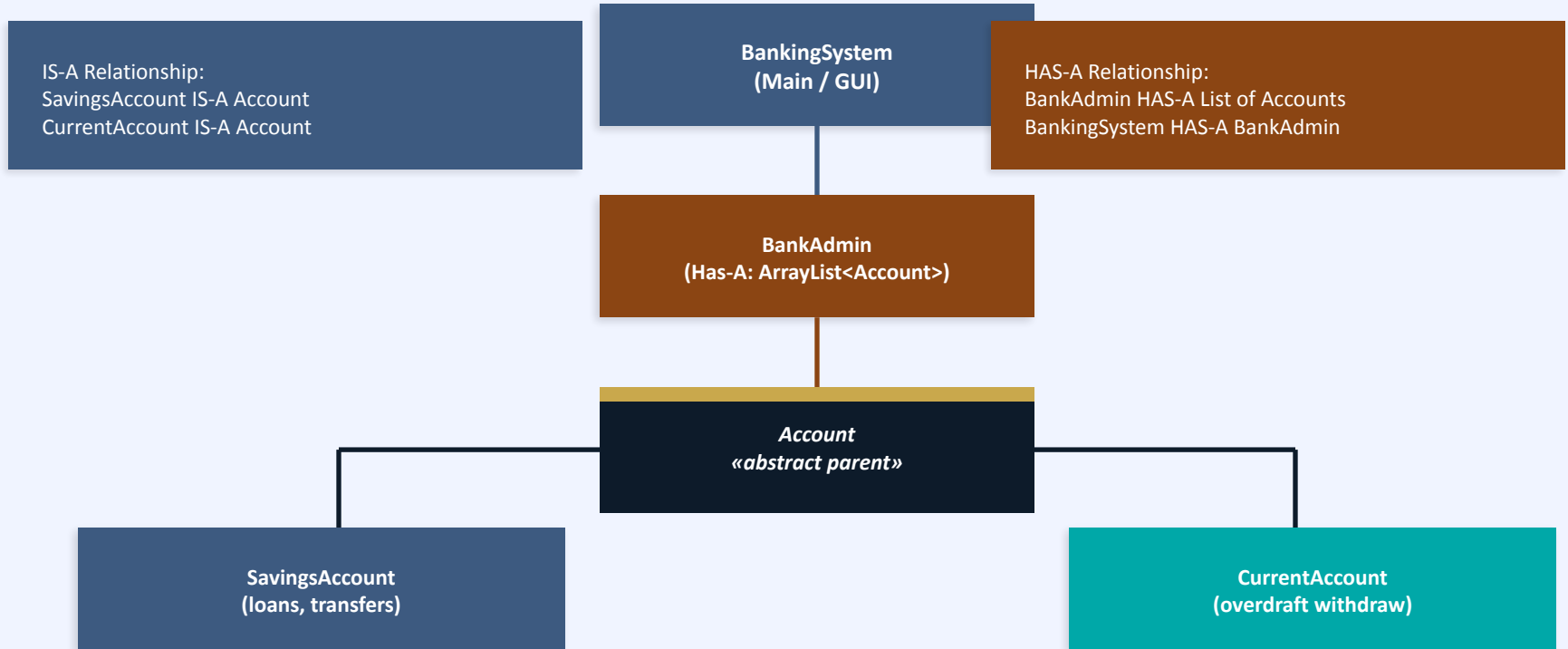
- ✓ Private fields: loanCount, loanEnabled, overdraftLimit
- ✓ Protected: accountHolderName, balance
- ✓ Public getters/setters control access
- ✓ BankAdmin encapsulates business logic



Abstraction

- ✓ Account is abstract — cannot be instantiated
- ✓ getAccountType() is abstract method
- ✓ Forces implementation in child classes
- ✓ GUI interacts only via BankAdmin interface

CLASS RELATIONSHIPS & DESIGN PATTERN



ADMIN PANEL — MANAGEMENT FEATURES

Create Account

Admin creates Savings or Current account; validates duplicate names; password confirmed twice

Delete Account

Confirm dialog before removal; removes from allAccounts ArrayList, savingsAccounts list, and passwordMap HashMap

View All Accounts

Iterates ArrayList; displays name, account number, type, and balance for every registered user

Bank Reserve

Tracks bankReserve (RM 100,000 initial); increases on deposit/loan repay; decreases when loan issued

Total Loans Issued

Cumulative totalLoanIssued counter; updated every time a user takes a loan from the system

Toggle Loan Feature

Admin enables/disables loan eligibility (loanEnabled boolean) per user by name; affects doLoan() check

09 | CONCLUSION

Project Achievements

- ✓ Successfully implemented Java Swing GUI Banking System
- ✓ Demonstrated inheritance: Account → SavingsAccount & CurrentAccount
- ✓ Applied runtime polymorphism via method overriding (getAccountType, withdraw)
- ✓ Applied compile-time polymorphism via method overloading (deposit)
- ✓ Used ArrayList & HashMap for account & password management
- ✓ Fixed transaction history bug — all transactions now timestamped correctly
- ✓ Loan repayment feature added with bankReserve tracking
- ✓ Admin panel fully functional with 6 management operations

Key Learnings

Class Design:

Abstract classes enforce consistent structure across child classes

Inheritance Hierarchy:

Code reuse through parent attributes and methods

GUI Programming:

Swing CardLayout enables smooth multi-screen navigation

Event-Driven Design:

Button listeners separate UI logic from business logic

Data Structures:

ArrayList + HashMap combination for efficient account management

Thank You!

Account Management System — CCC 1223 Group A

References

- [1] Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.
- [2] Deitel, P. & Deitel, H. (2020). Java: How to Program (11th ed.). Pearson.
- [3] Liang, Y. D. (2019). Introduction to Java Programming (12th ed.). Pearson.
- [4] Oracle Corp. (2024). Java SE Documentation. <https://docs.oracle.com/en/java/>
- [5] Oracle Corp. (2024). Java Swing Tutorial. <https://docs.oracle.com/javase/tutorial/uiswing/>



Group A

Md Ismayel Shekh

Kabisha Hafizi

Md Towhidul Islam
Tamim Mia

Md Ariful Islam